

Implementation and Analysis of BST, AVL and RBT Data Structures

Sebastiano Babich (172249) - Leonardo Mazzon (172587)
- Damiano Netto (171948)

University of Udine, Academic Year 2025 - 2026

Contents

1	Abstract	2
2	Introduction	2
3	Theoretical Background	2
3.1	Binary Search Tree (BST)	2
3.2	Adelson-Velsky Landis Tree (AVL)	3
3.3	Red Black Tree (RBT)	3
4	Implementation	4
5	Test Cases Setup	4
6	Results and Discussion	4
6.1	Test sugli inserimenti	4
6.2	Test sul numero di rotazioni	5
6.3	Test sull'altezza dell'albero	5
7	Conclusion	5
7.1	Confronto tra le strutture dati analizzate	5
7.2	Final Remarks	5

1 Abstract

The goal of the project was to implement and compare some of the data structures studied in the “Algorithms and Data Structures” course at the University of Udine, specifically standard and self-balancing binary search trees. The project focused in particular on the time complexity involved in inserting and deleting data within these structures. We also decided to analyze the heights reached by the trees during the various operations and to count the number of rotations performed on each tree.

2 Introduction

Search trees are among the most widely used data structures, since they enable high performance in both searching, inserting and deleting data. Specifically, the trees used in our project are binary: each node has at most two children, which can be other nodes or pointers to NULL. The resulting tree is acyclic, and one of its most important characteristics is its height that is defined as the maximum number of nodes on the path from the root to any leaf.

3 Theoretical Background

In all of the data structures which implements search trees, height is a fundamental property: keeping its value as small as possible helps ensure efficient operations thanks to more efficient search, insertion and deletion operations.

Trees can be divided into two categories based on how they manage their height: **binary search trees** and **balanced binary trees**. Standard binary search trees do not actively control their height, while balanced binary trees use specific rules and rotation operations to keep the height as small as possible.

Trees belonging to the second category are designed to maintain a height close to the order of $\log_2(n)$, where n is the number of nodes inserted into the tree.

In this project, we studied three types of trees, two of which implement self-balancing techniques.

3.1 Binary Search Tree (BST)

Binary search trees do not impose any restrictions on their height. Their main purpose is to provide efficient search, insertion and deletion operations maintaining an ordered structure of the stored elements. However, since they do not include any mechanism to control their height, the tree can become unbalanced depending

on the order in which nodes are inserted. As the number of nodes increases, this can lead to a significant degradation in performance, especially in the worst-case scenario.

Operation	Average Case	Worst Case
Search of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Insertion of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Deletion of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

Table 1: Time complexity in BST operations

3.2 Adelson-Velsky Landis Tree (AVL)

AVL trees are a data structure that implements self balancing binary search trees. The search operation in the tree exactly mirrors the search in BST. Insertion initially uses the BST insertion method, with the difference that a self balancing operation is performed afterward.

Specifically, this operation aims to identify imbalances in the tree by comparing the heights of two child nodes. Specifically, the goal is to minimize the total height of the tree, so that search operations remain as close as possible to logarithmic complexity.

Operation	Average Case	Worst Case
Search of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insertion of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Deletion of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 2: Time complexity in AVL operations

3.3 Red Black Tree (RBT)

...

Operation	Average Case	Worst Case
Search of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insertion of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Deletion of a Node	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 3: Time complexity in RBT operations

4 Implementation

The data structures (BST, AVL and RBT) were implemented as separate modules in the project, with similar interfaces for insertion, deletion, and search. Each tree maintains additional information useful for analysis, such as current height and number of rotations. Balancing operations were separated into dedicated functions within each type of tree to comply with the rules imposed by the data structure used.

5 Test Cases Setup

Per l'impostazione dei casi di test abbiamo inizialmente pensato a quali potessero essere. For the test cases, we have divided the code into three separate files:

- *setup_test.py*:
- *test_insertion.py*:
- *test_rotations.py*:

6 Results and Discussion

6.1 Test sugli inserimenti

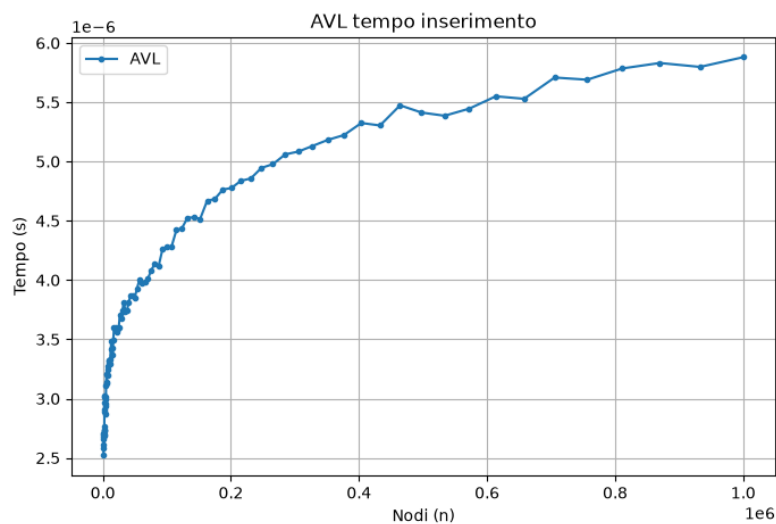


Figure 1: Test eseguito

6.2 Test sul numero di rotazioni

6.3 Test sull'altezza dell'albero

7 Conclusion

To conclude this project, we compared all the data structures we analyzed and drew conclusions regarding the ideal use cases for each type of tree.

7.1 Confronto tra le strutture dati analizzate

7.2 Final Remarks

The comparison demonstrates that the additional complexity required to implementing self-balancing trees is justified by the significant long-term gains in efficiency and robustness offered by these data structures.